PYTHON BASICS

Programming for Data Science

Topics

- Python Introduction
- Why Python
- Getting Started
- Code Writing Options
- Python Indentation
- Comments
- Variables
- Output Values
- Data types
- User Input

Python

- Python is a popular programming language.
- Python can be used on a server to create web applications. (Web Dev.)
- Python can be used alongside software to create workflows. (Soft. Dev.)
- Python can connect to database systems and perform operations.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.
- Python can also be used for system scripting.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.
- Python is a dynamic programming language.

Getting Started

- Python Install (https://www.python.org/downloads/)
- Use IDLE (Command Line, Python File)

```
C:\Users\Your Name>python helloworld.py
```

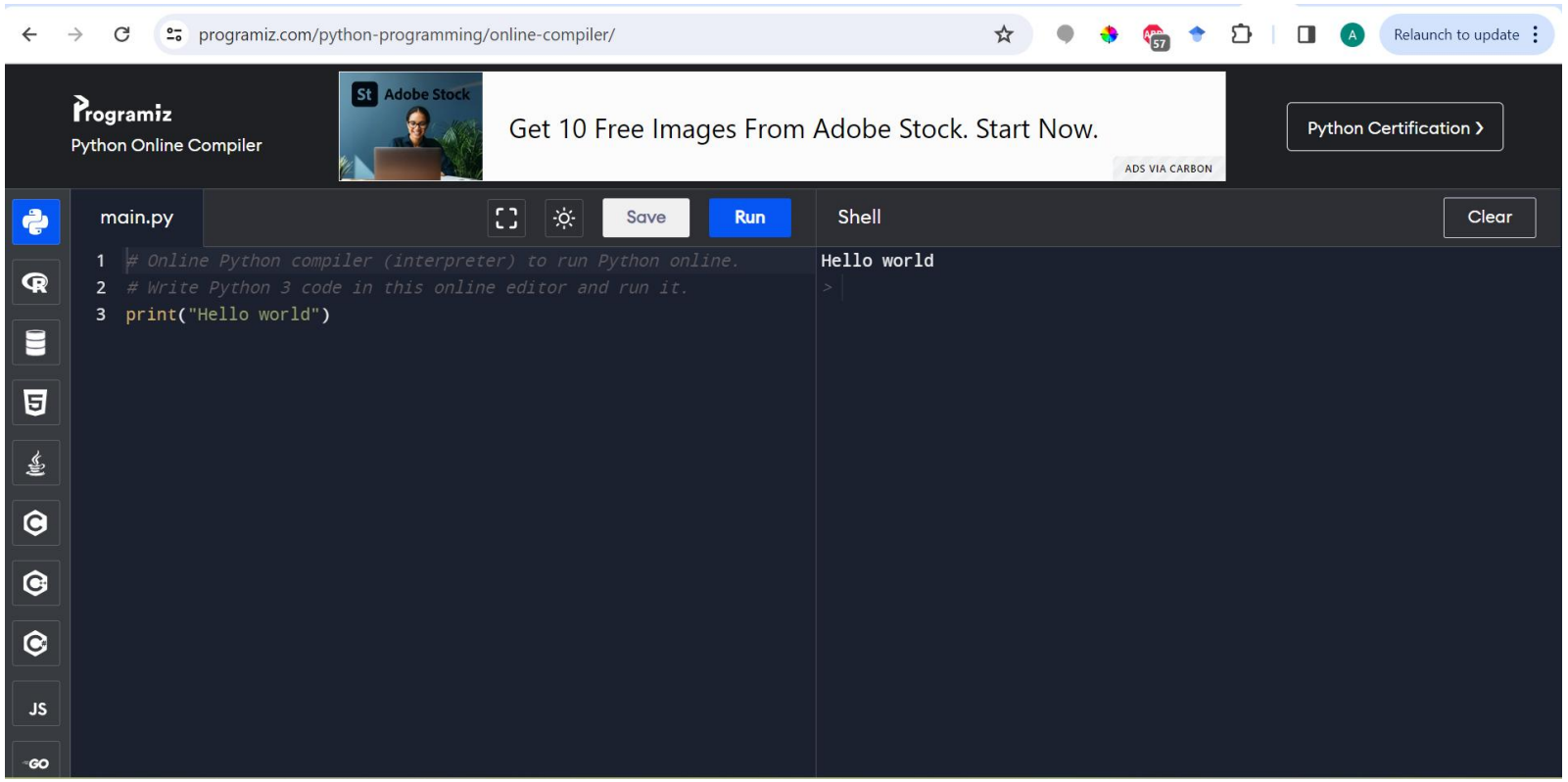
Using Windows command line

```
C:\Users\Your Name>python
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC
v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
```

Using Python command line

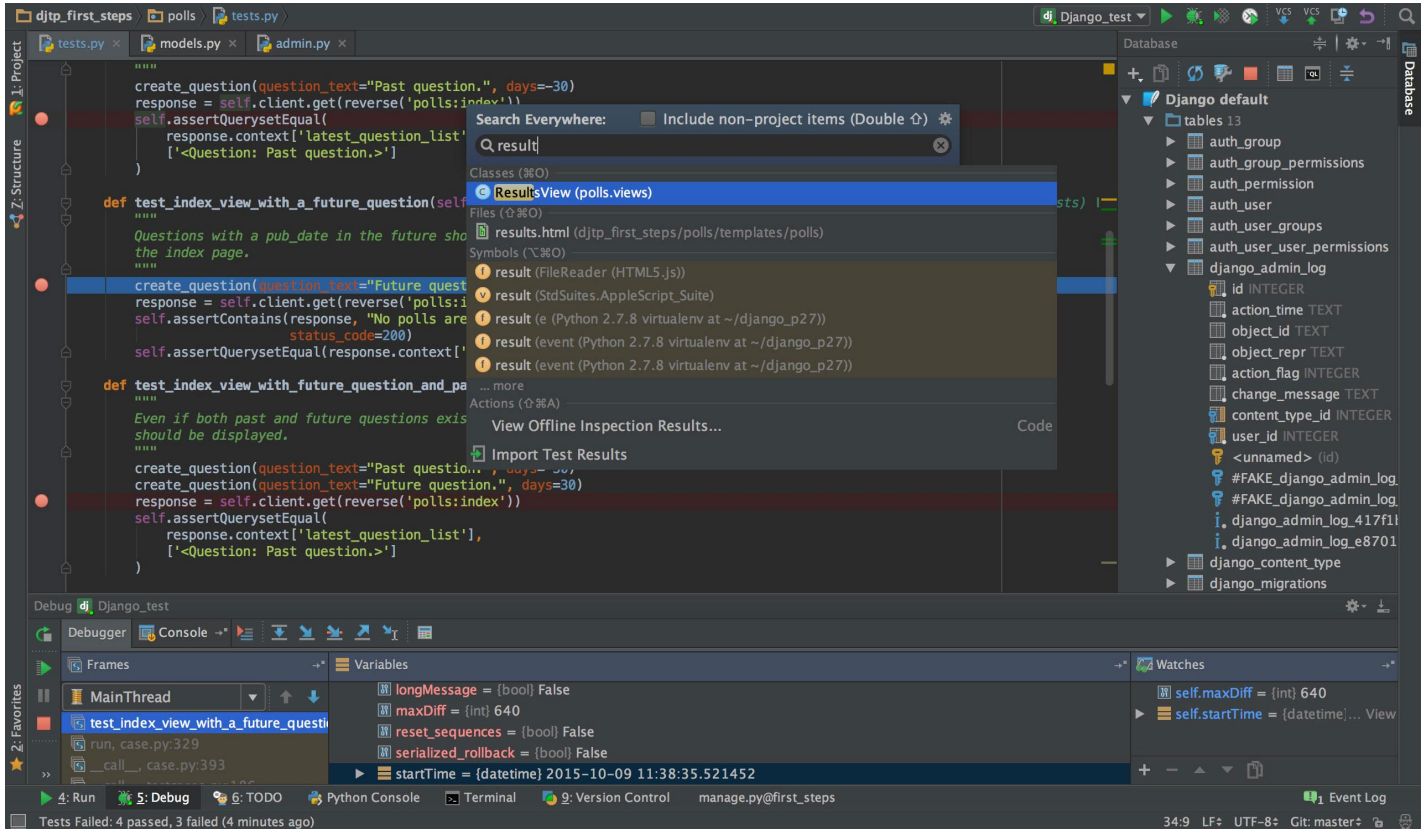
Code Writing Options

- Programiz - https://www.programiz.com/python-programming/online-compiler/



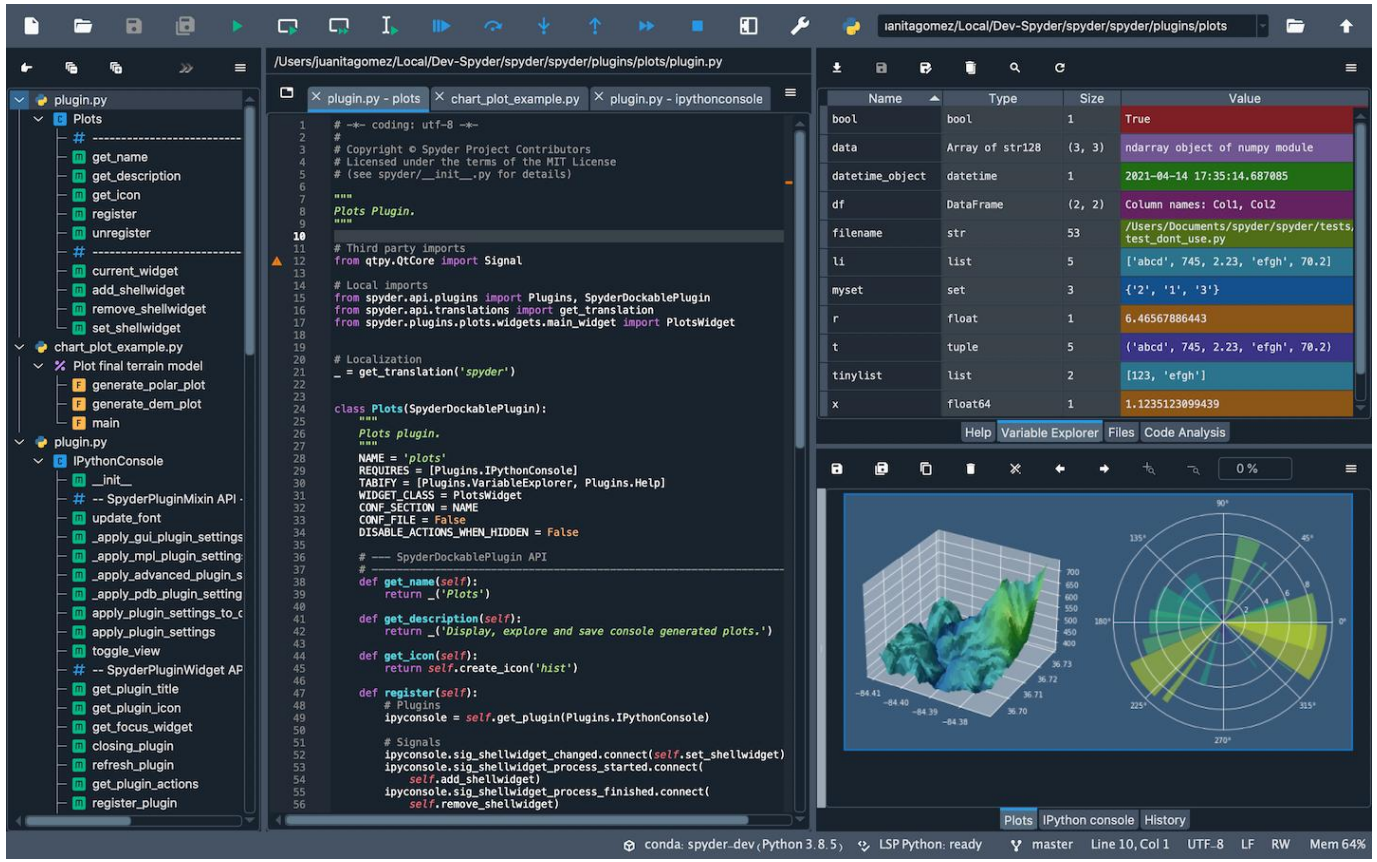
Code Writing Options

- PyCharm - https://www.jetbrains.com/pycharm/



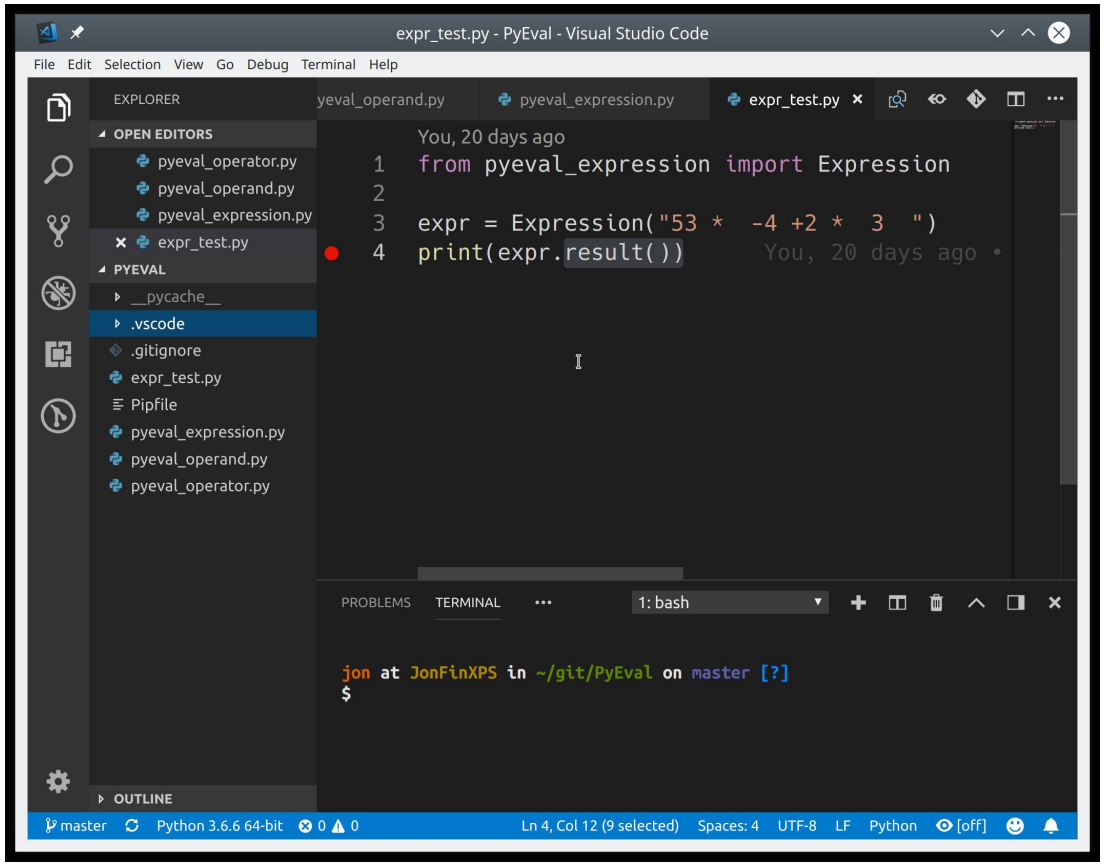
Code Writing Options

- Spyder - https://www.spyder-ide.org/



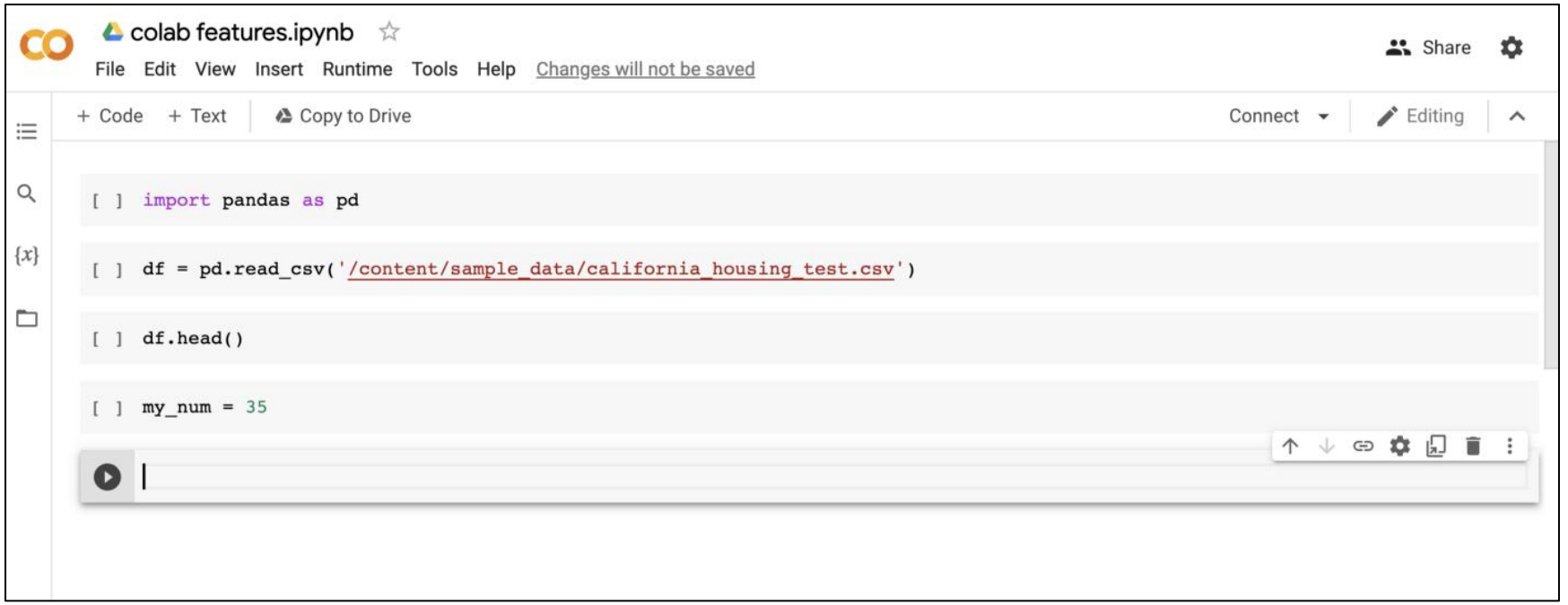
Code Writing Options

- Visual Studio Code - https://code.visualstudio.com/docs/languages/python



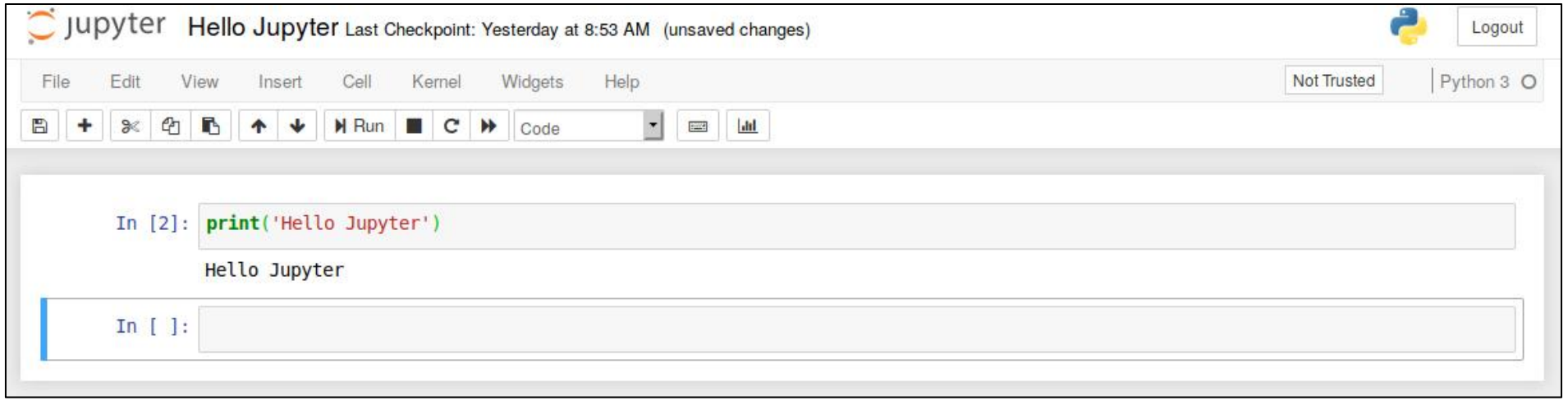
Code Writing Options

- Google Colab - https://colab.research.google.com/
- https://edulabs.co.il/p6-Google-Colab-a-must-have-tool-for-Developer-and-Data-Scientist.html



Code Writing Options

- Jupyter Notebook - https://colab.research.google.com/
- https://realpython.com/jupyter-notebook-introduction/



Python Indentation

- Indentation refers to the spaces at the beginning of a code line.
- Python uses indentation to indicate a block of code.

```
if 5 > 2:
 print("Five is greater than two!")

if 5 > 2:
        print("Five is greater than two!")
```

Comments

- Comments starts with a “#”, and Python will ignore them.
- Comments can be placed at the end of a line, and Python will ignore the rest of the line.
- Since Python will ignore string literals that are not assigned to a variable, we can add a multiline string (triple quotes) in our code, and place our comment inside it.

```
"""
This is a comment written in more than just one line.
Double or single quotation can be used.
"""
print("Hello, World!") # at the end of a line

# end of program
```

Variables

- Variables are containers for storing data values.
- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.
- Variables do not need to be declared with any particular type, and can even change type after they have been set.
- If you want to specify the data type of a variable, this can be done with casting.

Variables

```
x = 5
y = "John"
print(x)
print(y)
```

```
x = 4        # x is of type int
x = "Sally" # x is of type str
print(x)
```

```
x = "John"
# is the same as
x = 'John'

# Var. names are case-sensitive.
a = 4
A = "Sally"
#'A' will not overwrite 'a'
```

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
print(type(x))
print(type(y))
Print(type(z))
```

Variable Names

- Rules for Python variables:
  - A variable name must start with a letter or the underscore character
  - A variable name cannot start with a number
  - A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
  - Variable names are case-sensitive (age, Age and AGE are three different variables)
  - A variable name cannot be any of the Python keywords.

Variable Names

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

```
2myvar = "John"
my-var = "John"
my var = "John"
```

```
# Multi Words Variable Names
myVariableName = "John"     # Camel Case
MyVariableName = "John"     # Pascal Case
my_variable_name = "John"   # Snake Case
```

Output Variables

- The Python `print()` function is often used to output variables.

```
x = "Python is awesome"
print(x)

x = "Python"
y = "is"
z = "awesome"
print(x, y, z)

x = "Python "
y = "is "
z = "awesome"
print(x + y + z)
```

Output Variables

```
x = 5
y = 10
print(x + y)

x = 5
y = "John"
print(x + y)

x = 5
y = "John"
print(x, y)
```

Variables - Assign Multiple Values

```
# Many Values to Multiple Variables
x, y, z = "Orange", "Banana", "Cherry"
print(x, y, z)

# One Value to Multiple Variables
x = y = z = "Orange"
print(x, y, z)

# Unpack a Collection
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x, y, z)
```

Make sure the number of variables matches the number of values, or else you will get an error.

If we have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking.

Global Variables

- Variables that are created outside of a function (as in all of the examples so far) are known as global variables.
- Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"

def myfunc():
  print("Python is " + x)

myfunc()
```

Global Variables

```
x = "awesome"

def myfunc():
  x = "fantastic"
  print("Python is " + x)

myfunc()

print("Python is " + x)
```

```
# Without this line
# the output is same
x = "awesome"

def myfunc():
  global x
  x = "fantastic"

myfunc()

print("Python is " + x)
```

Data Types

Text Type:	`str`
Numeric Types:	`int`, `float`, `complex`
Sequence Types:	`list`, `tuple`, `range`
Mapping Type:	`dict`
Set Types:	`set`, `frozenset`
Boolean Type:	`bool`
Binary Types:	`bytes`, `bytearray`, `memoryview`
None Type:	`NoneType`

Data Types

```
x = 5
print(type(x))
```

Example	Example (Specifying the data type)	Data Type
x = "Hello World"	x = str("Hello World")	str
x = 20	x = int(20)	int
x = 20.5	x = float(20.5)	float
x = 1j	x = complex(1j)	complex
x = ["apple", "banana", "cherry"]	x = list(("apple", "banana", "cherry"))	list
x = ("apple", "banana", "cherry")	x = tuple(("apple", "banana", "cherry"))	tuple
x = {"name" : "John", "age" : 36}	x = dict(name="John", age=36)	dict
x = {"apple", "banana", "cherry"}	x = set("apple", "banana", "cherry"))	set
x = True	x = bool(5)	bool

User Input

- Python uses the `input()` method to ask for a user input.
- The `input()` function returns the input value as a string.
- Optionally, we can show a message to the user while asking for an input.

```
username = input("Enter username:")
print("Username is: " + username)
print(type(username))
```

Questions?

Thank you!